

Project Documentation: Reacting in Real-Time: Emotes and Kafka in Action!

COMP.CS.510 - Advanced Web Development: Back End

Lotta Lappalainen: lotta.lappalainen@tuni.fi 150416667

Julius Kari: julius.kari@tuni.fi

Kasper Kivistö: kasper.kivisto@tuni.fi

Group: Bakkarit

Repo URL: <https://course-gitlab.tuni.fi/compcs510-spring2025/bakkarit>

Project Documentation: Reacting in Real-Time: Emotes and Kafka in Action!.....	1
1) Introduction	3
2) Project plan	3
3) System Architecture.....	4
4) Component Details.....	5
4.1 Emote generator.....	5
4.2 Kafka Topics.....	5
4.3 Server B: Aggregation + Control API	5
4.4 Server A: WebSocket Bridge	6
4.5 Frontend: Viewer Dashboard	6
5) Used technologies	6
6) How to test the system	7
7) Learning diary	7
8) Monitoring & Logging.....	7
9) Security Considerations	8
10) Appendix.....	8

1) Introduction

The goal of this project is to build a distributed system that enables live stream viewers to react in real-time to meaningful moments during a broadcast. At the same time, it provides broadcasters with tools to monitor, analyze, and customize how these reactions are presented and interpreted.

The system supports real-time emote reactions, which are sent by viewers and processed immediately. When multiple reactions occur in a short timeframe, the system detects and aggregates these bursts to highlight moments of high engagement. Viewers receive live updates through WebSocket connections, ensuring a dynamic and responsive experience. On the broadcaster side, a RESTful API offers control over stream interaction settings and access to analytics, allowing for deeper customization and insight into audience behavior.

2) Project plan

The initial timetable for the research, design and implementation of the architecture

Which group member(s) will be responsible for what

How much time each group member promises to group project work per week. Note down each member's promise for committed hours for this project

Group members main responsibilities:

Julius: Server b functionality

Kasper: Server a functionality and frontend

Lotta: Frontend and docker containerizing

Promised hours to group project per week:

Julius:

Kasper:

Lotta: 1-5 or as much as is needed

3) System Architecture

- **High-Level Diagram:**

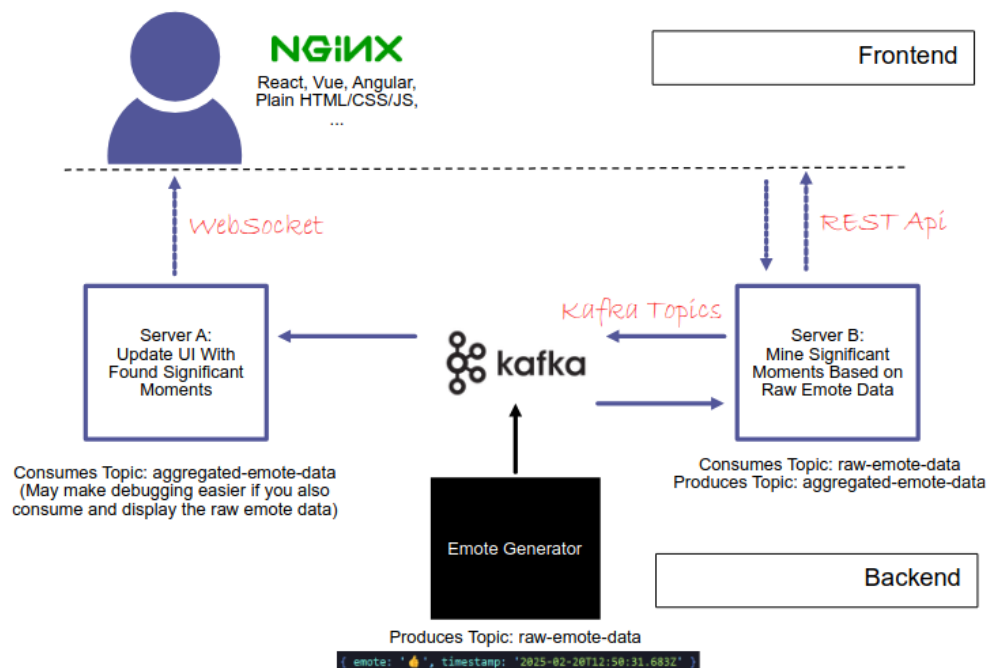


Image 1: High level diagram of the system. [1]

- **Components Overview:**

- Emote Generator
 - Writes new emote data every second
 - 80% chance for a single emote
 - 20% chance for a burst of multiple (same) emotes
 - Contains an emote and a timestamp as ISO string
- Kafka:
 - Transmits messages on two topics:
 - The raw emote data from the emote generator
 - The aggregated meaningful moments
- Server B (Aggregation + REST API)
 - Consumes the raw emote data and aggregates meaningful moments from it
 - Writes those to the meaningful moments' topic
 - Offers a REST API for checking and changing the meaningful moments threshold as well as checking and changing the collection of allowed emotes
- Server A (WebSocket Publisher)
 - Consumes the meaningful moments topic and writes it to the frontend using WebSocket connection

- Frontend Web App
 - Shows the aggregated data from the WebSocket connection
 - Can make asynchronous calls to server B to check and change the settings (needs some UI)

4) Component Details

4.1 Emote generator

- **Purpose:** Simulate viewer reactions for testing/initial input.
- **Behavior:**
 - Emits data every second.
 - 80% chance of a single emote.
 - 20% chance of a burst of identical emotes.
- **Output Structure:**

```
{
  "emote": "👉",
  "timestamp": "2025-04-06T10:32:11.123Z"
}
```

4.2 Kafka Topics

- **raw-emotes-topic:**
 - Source of truth for all incoming emote data.
 - Consumed by Server B for aggregation.
- **meaningful-moments-topic:**
 - Holds aggregated bursts of reactions.
 - Consumed by Server A to feed the frontend.

4.3 Server B: Aggregation + Control API

- **Responsibilities:**
 - Subscribes to raw-emotes-topic.
 - Detects and aggregates "meaningful moments" (e.g., high-frequency emote bursts).
 - Publishes aggregated results to meaningful-moments-topic.
- **REST API Endpoints:**
 - GET /settings/threshold → current meaningful moment detection threshold.

- PUT /settings/threshold → update threshold.
- GET /settings/emotes → list of allowed emotes.
- PUT /settings/emotes → update allowed emotes.
- GET /settings/interval → get current number of emotes to consume before analysis
- PUT /settings/interval → update interval
- **Aggregation Logic Example:**
 - Emotes in a short time window (e.g., 2 seconds) exceed a set threshold (e.g., 10).
 - Optional: Only count emotes from allowed list.

4.4 Server A: WebSocket Bridge

- **Responsibilities:**
 - Consumes messages from meaningful-moments-topic.
 - Forwards them in real-time to connected clients via WebSocket.

4.5 Frontend: Viewer Dashboard

- **Features:**
 - Live display of meaningful moments via WebSocket stream.
 - Interactive controls for broadcasters:
 - View/update threshold value.
 - Manage allowed emotes.
- **Tech Options:** React, Vue, or plain JavaScript.
- **UI Components:**
 - Live Emote Burst Feed.
 - Threshold Slider / Input.
 - Allowed Emotes Editor.

5) Used technologies

- **Tech Stack Overview:**
 - Backend: Kafka, Node.js, WebSocket server.
 - Frontend: React / Vue / JavaScript.
 - Communication: Kafka (messaging), REST (control), WebSockets (real-time updates).

6) How to test the system

// Varmaan turha tää alempi?

Tähän si mitkä komennot tms tarvitaan ohjelman suorittamiseen

Here group describes how course personnel can test their system. A complete HOWTO on running your system, with easily copy-pastable examples.

- Recommended environment layout (Docker/Kubernetes if used).
- Environment variables and settings for:
 - Kafka brokers
 - WebSocket server
 - REST endpoints
- Unit and integration tests for:
 - Emote generation
 - Aggregation logic
 - REST API
- Frontend testing:
 - WebSocket connection simulation
 - UI interaction with backend

7) Learning diary

A group learning diary, where group members note down the learning during the project. Short descriptions of who, what, and when are sufficient. Could be a link to GitLab Issue, that has more information.

This information is interesting to the course personnel, too. We get to see if the project enabled learning as designed.

8) Monitoring & Logging

// Turha varmaan kans ja alempi

- Log meaningful events in each component (e.g., emote bursts, API updates).
- Optional: Integrate Prometheus/Grafana for real-time metrics.

9) Security Considerations

- Rate-limiting REST endpoints.
- Input validation.
- Optional: Authentication layer for broadcasters.

10) Appendix

- Glossary:
 - *Emote*: A viewer's reaction symbol.
 - *Burst*: A rapid series of identical emotes.
 - *Meaningful Moment*: A burst that passes a reaction threshold.
- Example payloads.
- References to libraries/tools used.